Chat com ordenação

Documento de protocolo – v2

Levi Henri André Gabriel

6 de novembro de 2026

Estado. A v2 fecha o contrato da operação normal, do heartbeat, do Bully, do bootstrap e da reorganização após uma queda (seções 2 a 4), implementados em `app/` e testados (seção 5). A reentrada de um nó reiniciado continua como proposta, para a T06. O texto ainda depende da revisão do grupo. Arquitetura e justificativas ficam em `docs/relatorio.tex`.

1 Escopo e transporte

Três nós equivalentes, identificados por 1, 2 e 3, trocam JSON por HTTP. Clientes usam WebSocket. Nomes dos serviços vêm da configuração e são resolvidos pelo DNS do Compose; não há IPs fixos no código. O cenário inicial é uma sala, com estado em memória.

A recuperação proposta pressupõe uma queda por vez, rede funcionando entre os sobreviventes e nenhuma nova falha durante reorganização ou sincronização. São necessários dois dos três nós para operar. O relatório detalha os limites.

2 Operação normal

Identidade e visão. Cada nó tem ID 1, 2 ou 3. Uma visão contém época, líder e membros ativos. A época é proposta como o par (contador, ID do coordenador), que distingue propostas sem substituir acordo entre participantes. Estados locais: sincronizando, ativo e reorganizando.

Cada nó mantém relógio L, fila de pendências, ACKs recebidos, histórico, UUIDs entregues e identificação dos pacotes já processados. O histórico preserva o conteúdo, a época e o carimbo original de cada evento, necessários para a reconciliação; guardar apenas UUIDs não basta.

Cliente. Envia `id`, `sala`, `autor` e `texto`; `id` é um UUID preservado nos reenvios. Reutilizar um UUID com conteúdo diferente é erro de protocolo.

Envelope entre nós. Campos comuns: `tipo`, `pacote_id`, `epoca` e `origem`. Um retry preserva o mesmo pacote. `MSG` acrescenta relógio e conteúdo da mensagem; `ACK` acrescenta seu relógio e a referência ao evento reconhecido. O evento é identificado pela época, origem e timestamp original.

1. Ao aceitar uma mensagem do cliente, o nó incrementa L, registra o evento e enfileira seu multicast para todos os membros, incluindo recepção local.
2. Ao receber `MSG`, atualiza $L \leftarrow \max(L, t) + 1$ e guarda a mensagem na fila, ordenada pelo **carimbo original** (t, origem). O relógio atualizado do receptor não altera essa chave.
3. O receptor incrementa seu relógio para emitir `ACK` a todos os membros. Receber um ACK também atualiza L pela regra de recepção. ACKs que chegam antes do MSG correspondente são guardados.
4. A primeira mensagem da fila só é entregue após ACK de todos os membros daquela visão. O UUID é registrado no histórico e os clientes locais são avisados.

Confirmação. O nó do cliente confirma a mensagem quando a entrega. Receber pelo WebSocket ou responder a um POST não confirma entrega. Na reconexão, o cliente reenvia usando o mesmo UUID; o reenvio também serve de consulta. O contrato com o navegador está na seção 4.

Relógio. Modela os eventos da camada `MSG/ACK`. Heartbeats ficam fora dessa ordenação. Dentro de uma visão, a chave é (t, origem); entre visões, o histórico anterior deve permanecer como prefixo imutável. Lamport ordena mensagens; o líder coordena a visão.

FIFO, retries e deduplicação

Uma fila de saída por destino envia um POST por vez e só avança após a resposta. Falhas de transporte repetem o mesmo pacote. Uma resposta pode se perder depois do processamento: o receptor deve responder ao retry sem repetir seus efeitos.

Atualização de estado e enfileiramento de cada multicast são serializados. O multicast inteiro é enfileirado antes de processar outro evento. O handler não espera respostas de outros nós para concluir o processamento local.

A deduplicação ocorre no transporte e na entrega. Pacotes repetidos não geram novo evento nem nova emissão de ACK; um UUID entregue não reaparece no histórico.

Identidade após reinício. O `pacote_id` tem a forma `<origem>-<encarnação>-<sequência>`. A encarnação é sorteada a cada início do processo; um nó reiniciado nunca repete um identificador antigo.

Mesmo UUID em nós distintos. Se o cliente reenviar a mesma tentativa a outro nó antes da entrega, podem existir dois eventos com o mesmo UUID. Ambos seguem a ordem total, e todos os nós entregam somente a primeira ocorrência, descartando as seguintes na entrega. Como todos veem a mesma sequência, todos descartam a mesma ocorrência. Dentro de um nó, UUID repetido com conteúdo igual não cria novo evento; com conteúdo diferente é recusado.

Heartbeat e eleição

Parâmetros padrão: heartbeat a cada 1 segundo e suspeita após 3 segundos sem heartbeat; espera de `OK` de 1 segundo e de `COORDINATOR` de 3 segundos. São configuráveis e não representam prazo garantido de recuperação. Heartbeats vão a todos os nós configurados, inclusive fora da visão.

A suspeita de qualquer membro passa o nó a *reorganizando*: novos envios de clientes são recusados, enquanto MSG e ACK da visão atual continuam sendo processados até a pausa da transição. Com líder disponível, ele coordena a reorganização. Se o coordenador for suspeito, os membros com visão executam Bully com `ELECTION`, `OK` e `COORDINATOR`: `ELECTION` vai aos membros de ID maior não suspeitos; sem `OK`, o nó se anuncia; com `OK` e sem anúncio dentro do prazo, repete a eleição. Quem recebe `ELECTION` de ID menor responde `OK` e inicia a própria eleição. Um anúncio de ID menor recebido por um nó vivo de ID maior provoca nova eleição.

Mensagens de eleição de outra época são ignoradas. Nós sincronizando não respondem nem votam. Maiores que não respondem ao `ELECTION` dentro do prazo passam a suspeitos. Vencer só define o coordenador da transição: a visão e a época permanecem, e os envios continuam pausados até a retomada da seção 3. Suspeitas valem até a próxima instalação de visão.

3 Reorganização após uma queda (T05)

Hipóteses. Grupo inicial de três nós; uma queda por vez; sobreviventes conectados entre si; nenhuma nova falha durante a transição. Só há retomada com pelo menos dois dos três nós configurados. Fora disso, os nós ficam pausados.

Quem coordena. Na queda do líder, o Bully elege o maior ID vivo. Na queda de um membro comum, o líder continua. Nos dois casos, o coordenador é o maior ID entre os membros não suspeitos, e só ele propõe. A proposta é única para aquela recuperação: época $(c + 1, \text{coordenador})$, em que c é

o contador da época encerrada, e membros iguais à visão menos os suspeitos. Ser eleito não libera envios; só a barreira abaixo retoma o chat.

Pausa e corte. A suspeita pausa apenas os envios de clientes: MSG e ACK da visão continuam sendo processados. O corte vem com `VIEW_PREPARE`. O participante confere se a proposta encerra a sua visão, se ele está entre os membros, se os membros pertencem à visão, se a origem é o maior deles e coincide com o coordenador da época proposta, se a época é maior e se há maioria. Aceita, no mesmo passo síncrono (sem `await`), ele pausa, captura histórico publicado, pendências e relógio, encerra os MSG/ACK da época antiga em todos os canais, inclusive para o nó removido, e responde `VIEW_STATE`. Um PREPARE repetido recebe a mesma captura. Depois do corte, MSG e ACK da visão encerrada recebem `200` com `descartado` (*visão encerrada*), sem efeito algum.

Reconciliação. Com o `VIEW_STATE` de todos os membros propostos, inclusive o seu, o coordenador:

1. exige que os históricos publicados sejam prefixos uns dos outros e usa o mais longo como base, preservando-o inteiro, com as visões anteriores;
2. acrescenta depois da base as pendências da época encerrada que ela ainda não contém, pela chave original (relógio, origem), sem esperar o ACK do nó caído;
3. deixa de fora uma pendência cujo UUID já ocorreu antes na ordem total: vale a primeira ocorrência, a mesma decisão da entrega normal;
4. usa como relógio de referência o maior relógio informado.

Com FIFO e ACK de todos, nenhuma pendência *nova*, de UUID ainda não ocorrido, pode ter chave menor que a da última mensagem entregue naquela visão. Uma duplicata já resolvida pode: se o nó 1 entregou $A(1,1)$, descartou a outra ocorrência $A(6,2)$ e entregou $B(11,1)$, o nó 2 ainda pode ter $A(6,2)$ pendente. Por isso o UUID é conferido antes da ordem. Se uma pendência nova anteceder uma entrega, se uma ocorrência anteceder a primeira já confirmada, se os históricos divergirem ou se um mesmo evento vier com conteúdos diferentes, nada é reescrito: o erro aparece em `/estado` e no log, e os nós continuam pausados.

Instalação e retomada. `VIEW_INSTALL` leva o histórico reconciliado. Cada membro confere que o que já publicou é prefixo dele, instala a visão sem reduzir o relógio, permanece pausado (fase *instalada*) e responde `VIEW_READY`. Com todos os READY, o coordenador retoma e envia `VIEW_RESUME`, reenviando até ver todos ativos pelos heartbeats. Na retomada, cada nó publica só o sufixo que ainda não tinha entregue, uma vez e nas posições seguintes. Antes disso, `/historico` e o histórico inicial do WebSocket mostram apenas o já publicado, e dados da nova época recebem `503`.

Controles perdidos são reenviados a cada ciclo de heartbeat. Controles repetidos, atrasados ou de outra transição recebem `200` com `ignorado`: não reinstalam, não reabrem transição concluída, não liberam envios nem repetem notificações. Durante a transição o nó não vota, mas responde `OK` a um `ELECTION` de ID menor, para não ser tomado como falho.

Nó reiniciado. Volta sincronizando, sem visão e com outra encarnação. Cada nó guarda a encarnação vista em qualquer pacote de cada par, tirada do `pacote_id`, inclusive no bootstrap. Um membro em operação que aparece com outra encarnação conta como falha, mesmo que reinicie logo após o bootstrap, antes de anunciar a época. Pacotes normais ou atrasados da mesma encarnação, como um heartbeat do bootstrap com época `null`, nunca parecem um reinício. O nó fica fora da visão e da eleição até a admissão por `JOIN`/`SNAPSHOT` (T06).

Exemplo: mensagens em trânsito

A visão inicial é época $(1,3)$, membros $\{1,2,3\}$ e líder 3. Os históricos já contêm a mensagem `oi`. O nó 1 origina `x`, com carimbo $(10,1)$; o nó 3 origina `y`, com carimbo $(20,3)$, e cai depois de enviar cópias.

Antes das pausas, o nó 1 tem todos os ACKs de `x` e a entrega. O nó 2 recebeu o MSG de `x`, que vale como reconhecimento do nó 1, mas não o ACK do nó 3. Esse ACK e a cópia de `y` destinada ao nó 2 seguem em trânsito, nessa ordem, no canal 3→2, como exige o FIFO. A cópia de `y` para o nó 1

chegou.

Estado capturado no corte	Nó 1	Nó 2
Histórico entregue	`[oi, x]`	`[oi]`
Pendências conhecidas	`[y]`	`[x]`

O Bully escolhe o nó 2 entre os sobreviventes. Ambos cortam e informam os estados acima. Quando o ACK antigo e a cópia de `y` chegam ao nó 2 depois do corte, recebem `200` de descarte e não alteram a captura nem produzem entrega. O conteúdo de `y` já está no relato do nó 1 e é recuperado pela transição.

A união ordenada dos relatos produz `[oi, x, y]`, que estende os dois históricos. A proposta instala a visão de época $(2, 2)$, membros $\{1, 2\}$, líder 2 e esse histórico. Após os dois READYs, a retomada entrega apenas `y` aos clientes do nó 1 e `x, y` aos do nó 2. As pendências recuperadas são resolvidas pela instalação comum, sem esperar ACK do membro removido. A instalação pode recuperar uma mensagem que ainda não tinha sido confirmada; ausência de confirmação não significa cancelamento.

Uma mensagem `z` conhecida somente pelo nó 3 que caiu pode não aparecer na coleta. Ela não poderia ter sido confirmada pelo fluxo normal sem ACK dos demais membros. Seu cliente poderá repetir a tentativa com o mesmo UUID. Este exemplo não cobre falhas adicionais durante a reorganização. Ele é um teste automático (`tests/test_reorganizacao.py`), montado com essa sequência FIFO.

Reentrada: proposta para a T06

O nó retorna sincronizando e envia `JOIN`. Sua admissão usa a mesma pausa. Recebe `SNAPSHOT` com histórico, UUIDs entregues, visão e relógio antes de entrar no grupo ativo. Depois pode participar de uma eleição Bully, inclusive se tiver o maior ID.

4 Contrato v2

Rotas. Entre nós: `POST /interno/pacote`, corpo JSON com o envelope. Clientes: WebSocket em `/ws`. Observação: `GET /estado` e `GET /historico`. Época é a lista `[contador, coordenador]`, ou `null` enquanto o nó não tem visão. Todos os pacotes levam `tipo`, `pacote_id`, `epoca` e `origem`.

Tipo	Campos além do envelope e função
`MSG`	`relogio` (carimbo original), `mensagem` {id, sala, autor, texto}.
`ACK`	`relogio` (do emissor), `evento` {origem, relogio}; época do envelope.
`HEARTBEAT`	`estado`, `coordenador`. Fora do relógio lógico.
`ELECTION` / `OK`	Sem campos extras. Iniciar eleição e responder como ID maior.
`COORDINATOR`	Sem campos extras: o vencedor é a `origem`. Não libera envios.
`VIEW_PREPARE`	`transicao`, `anterior`, `membros`: propõe a visão e pede o corte.
`VIEW_STATE`	`transicao`, `anterior`, `historico`, `pendentes`, `relogio`: a captura do corte.
`VIEW_INSTALL`	`transicao`, `anterior`, `lider`, `membros`, `historico` e `relogio`; no bootstrap, `anterior` é `null`.
`VIEW_READY`	`transicao`: confirma a instalação.
`VIEW_RESUME`	`transicao`: libera a operação após todos os READY.
`JOIN` / `SNAPSHOT`	A definir em T06: admissão e transferência de estado.

A referência de transição (`transicao`) é a época proposta, e `anterior` é a época encerrada; juntas identificam a transição mesmo depois que a época do envelope muda com a instalação. Históricos e pendências são listas de eventos `{epoca, relogio, origem, mensagem}`, com o carimbo original.

Respostas HTTP. `200`: processado, repetido (`duplicado`), descartado por época antiga ou ignorado; em todos os casos, o retry termina. `503`: o receptor ainda não pode processar (sem visão, visão instalada aguardando `VIEW_RESUME` ou época mais nova que a sua); o canal repete o mesmo pacote

sem avançar, preservando o FIFO. Depois do corte, MSG/ACK da visão encerrada recebem `200` de descarte, e o próprio emissor já os retirou das suas filas. Falha de transporte, timeout e outros 5xx também repetem o mesmo pacote. `400`: pacote malformado, recusado antes de qualquer efeito. O envelope e os campos do tipo são conferidos primeiro; números são inteiros JSON, sem booleanos, e só os controles aceitam época `null`. Uma recusa como essa indica defeito: o pacote não é repetido, e o canal daquele destino para nele, registrando o motivo, sem deixar o seguinte ultrapassá-lo. Os outros destinos seguem. Só MSG e ACK passam pelo canal FIFO com retry e pela deduplicação por `pacote_id`. O controle é enviado uma vez, de forma direta, e é idempotente: quem precisa de resposta o reenvia a cada ciclo de heartbeat.

Bootstrap. Todos começam *sincronizando*, sem visão. O nó de maior ID configurado propõe a visão $(1, \text{ID})$ com todos os nós configurados quando recebeu heartbeat recente de cada um e nenhum tem visão. Ele envia `VIEW_INSTALL`; cada nó instala, responde `VIEW_READY` e segue pausado; com todos prontos, o coordenador retoma e envia `VIEW_RESUME`, reenviando até ver todos ativos pelos heartbeats. Logo, o grupo inicial exige os três nós. Um nó que reinicia encontra pares com visão, não faz bootstrap e aguarda admissão por `JOIN` (T06). Reiniciar todos começa um grupo novo e vazio.

Cliente. O cliente envia `{id, sala, autor, texto}`. O nó envia, na conexão, `{tipo: historico, no, mensagens}`. A cada entrega, envia `entregue` com `posicao`, `epoca`, `relogio`, `origem` e `mensagem`; esse aviso é a confirmação. Respostas imediatas: `aceita` (em andamento, não é confirmação), `confirmada` (UUID já entregue) ou `recusada` com `motivo` (campos inválidos, UUID reutilizado com outro conteúdo ou nó fora do estado ativo). Tentativa pendente ou recusada é reenviada com o mesmo UUID.

5 Decisões ainda abertas e validação

Para a T06: formato de `JOIN` e `SNAPSHOT` e a admissão de um nó reiniciado. Ainda sem tratamento: limpeza dos identificadores de pacotes processados (guardados sem limite durante a execução), falhas novas durante uma transição (inclusive do coordenador) e partições.

Validação da v1 (`tests/`): uma simulação em memória intercala canais ao acaso, mantém FIFO em cada canal, repete pacotes e responde 503 a um nó ainda pausado. Em 200 sementes, os três nós entregaram a mesma sequência, sem perdas nem duplicatas. Defeitos injetados de propósito (entregar com ACKs parciais ou ordenar pelo relógio do receptor) fizeram a simulação falhar. Testes assíncronos cobrem Bully, suspeita, bootstrap e a exclusão de nós sincronizando. Um teste de integração sobe três processos reais em `localhost`, envia 45 mensagens concorrentes por três clientes WebSocket e derruba o líder; os sobreviventes elegeram o nó 2 e mantiveram os envios pausados. Em 6/11, testes do canal FIFO real com HTTP simulado cobriram 500, 503, resposta perdida e recusa permanente, e 37 pacotes malformados foram recusados sem efeitos. No Compose, os três nós entregaram as mesmas 60 mensagens concorrentes e, após a queda do nó 3, elegeram o nó 2 e permaneceram pausados. Esses testes não provam a correção geral.

Validação da T05 (6/11): 345 testes passaram, em três execuções da suíte depois das correções da revisão. As regressões dessas correções falharam antes delas: duplicata já resolvida bloqueando a reconciliação, reinício logo após o bootstrap sem detecção e heartbeat atrasado do bootstrap tomado por reinício. Na simulação, com o Bully real, a queda do líder no meio do tráfego terminou na visão $(2, 2)$ com históricos iguais, prefixos preservados e mensagens novas entregues, em 15 sementes da suíte e em 300 de uma rodada avulsa. A queda de membro comum terminou em $(2, 3)$ com o líder 3. Também foram cobertos: o exemplo acima com a sequência FIFO, a perda de cada controle uma vez e o reenvio de todos eles depois da retomada sem efeito, MSG/ACK antigos após o corte, dados novos com 503 até o RESUME, históricos incompatíveis (pausa com erro), UUID repetido em nós distintos, relógio que não diminui, um sobrevivente sem retomada, nó reiniciado fora da visão e clientes que não recebem o sufixo antes do RESUME. No teste de integração local, a queda do líder terminou com os nós 1 e 2 ativos em $(2, 2)$ e uma mensagem nova entregue. No Compose (`scripts/validar_compose.py`, três execuções, a última depois das correções), 45 mensagens concorrentes precederam a queda do nó

3 e, em outro cenário, do nó 1. A nova visão ficou ativa de 4 a 5 segundos após o `kill`, com históricos preservados e uma mensagem nova entregue aos dois sobreviventes. Nesses dois cenários, o tráfego já tinha terminado quando o nó caiu: a recuperação de mensagens em trânsito foi exercitada só na simulação.

Referências iniciais

Enunciado do professor: Trabalho Final – Sistemas Distribuídos 2026, fornecido durante o planejamento. Lamport (1978); Garcia-Molina (1982); Van Renesse et al., comunicação e mudança de visão.